The best core architectural style for quick entry to IT business is going to be a **Modular Monolith**

Modular monolith style is perfect to create a single deployable application with strictly separated internal modules. It requires less programmers and workers and its simpler to test and minimizes network overhead.

For example, lets assume a group of 3-4 junior developers have a startup idea called **"EduFlow".** It' a platform for local schools, universities and local teaching companies to register their students, help them create their schedule and track their activity, transaction and payment. The main goal of a business is to get a quick entry to IT market and get an attention of first 50 paying users within 2-3 month. Platform has 4 internal modules:

1) User/Auth Module – for registration of students and teachers
2) Scheduling Module – calendar and class schedule
3) Billing Module – track the payment and save story of payment
4) Notification Module – sending messages and notifications about classes

Why exactly Modular Monolith in this case? Because its cheap and very fast to create. All 4 modules are in 1 repository and its easy to maintain all of them. Also, it's easier to test since platform is not big.

The team consists of junior, mid-level developers. Platform doesn't require a big server yet, therefore monolith architecture gives developers an opportunity to focus on writing the business logic, not complex conceptions (like distributing the system)

But, here are **Trade-Offs** of Modular Monolith:

What we consciously sacrifice for the sake of speed: 1) Single Point of Failure. If 1 module has a big bug or error(For example Notification module) then this module will continuously send the messages to student or teachers and it will become impossible to normally use the whole platform. Every module is deeply connected to each other so its important to keep all of them save in order to not brick the whole system.

2) Limitations for scaling. If for example business will grow within a year and the number of users will also grow, then its gonna be hard or almost impossible to scale platform. Growth of business will require changing of modules and more resources(For example servers with the large amount of memory)

In conclusion, we consciously sacrifice scalability, complex connections between modules to get faster entry to IT business and for lower costs of resources. When the platform becomes larger, it will definitely require big changes in code and whole structure.